AI Banking Assistant - Test Execution Guide

Prerequisites & Step-by-Step Test Instructions

Part 1: Prerequisites (Complete Before Testing)

Complete ALL of the following prerequisites before running any tests. These steps ensure your environment is properly configured.

1.1 Software Requirements

- ❏ Python 3.9+ installed (verify: python3 --version)
- ❏ pip package manager available (verify: pip --version)
- ❏ curl installed for API testing (verify: curl --version)
- ❏ AWS CLI installed and configured (verify: aws --version)
- ❏ Git installed (optional, for version control)

1.2 AWS Account & Credentials

- ❏ Active AWS account with billing enabled
- ❏ IAM user or role with DynamoDB and Bedrock permissions
- ❏ AWS credentials configured locally

Configure credentials:

```
aws configure
# Enter:
#   AWS Access Key ID: <your-key>
#   AWS Secret Access Key: <your-secret>
#   Default region: us-east-1
#   Output format: json
```

Verify credentials work:

```
aws sts get-caller-identity
```

Expected: Returns your Account ID, ARN, and UserId without errors

1.3 Create DynamoDB Contracts Table

Create the table via AWS CLI:

```
aws dynamodb create-table \
  --table-name Contracts \
  --key-schema AttributeName=contract_id,KeyType=HASH \
  --attribute-definitions AttributeName=contract_id,AttributeType=S \
  --billing-mode PAY_PER_REQUEST \
  --region us-east-1
```

Insert test data:

```
aws dynamodb put-item \
  --table-name Contracts \
  --region us-east-1 \
  --item '{
    "contract_id": {"S": "C123"},
    "amount": {"N": "50000"},
```

```
    "interest_rate": {"N": "0.05"},
    "duration": {"S": "5 years"}
  }'
```

Verify the item was inserted:

```
aws dynamodb get-item \
  --table-name Contracts \
  --region us-east-1 \
  --key '{"contract_id": {"S": "C123"}}'
```

Expected: Returns the full item with contract_id, amount, interest_rate, duration

AI Banking Assistant - Test Execution Guide

Prerequisites & Step-by-Step Test Instructions

1.4 Enable Bedrock Model Access

- ❏ Go to AWS Console > Amazon Bedrock > Model access
- ❏ Click 'Manage model access'
- ❏ Enable: Anthropic > Claude 3 Haiku
- ❏ Click 'Save changes' and wait for status = 'Access granted'

NOTE: Model access can take 1-5 minutes to activate. Verify status shows 'Access granted' before proceeding.

Verify Bedrock access via CLI (optional):

```
aws bedrock list-foundation-models --region us-east-1 --query 'modelSummaries[?modelId==`anthropic.clau
```

Expected: Returns ["anthropic.claude-3-haiku-20240307-v1:0"]

1.5 Install Project Dependencies

```
cd /Users/k.nayak.pradeep/Downloads/Kirodemo

# Install runtime dependencies
pip install -r requirements.txt

# Install test dependencies
pip install -r requirements-dev.txt
```

Verify installation:

```
python -c "import fastapi, boto3, pydantic, uvicorn; print('All dependencies OK')"
python -c "import pytest, hypothesis, httpx, moto; print('Test dependencies OK')"
```

Expected: Both commands print 'OK' without import errors

1.6 Verify IAM Permissions

Minimum permissions required:

```
# DynamoDB
dynamodb:GetItem on arn:aws:dynamodb:us-east-1:*:table/Contracts

# Bedrock
bedrock:InvokeModel on arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-3-haiku-20240307-v1
```

Prerequisites Completion Checklist

- ❏ Python 3.9+ installed
- ❏ AWS credentials configured (aws sts get-caller-identity works)
- ❏ DynamoDB 'Contracts' table created in us-east-1
- ❏ Test record C123 inserted into Contracts table
- ❏ Bedrock Claude 3 Haiku model access enabled
- ❏ pip install -r requirements.txt completed
- ❏ pip install -r requirements-dev.txt completed

AI Banking Assistant - Test Execution Guide

Prerequisites & Step-by-Step Test Instructions

Part 2: Test Execution Steps

Follow these steps in order. Each step builds on the previous one.

Step 1: Run Unit Tests (Offline - No AWS Needed)

Unit tests use moto (AWS mock library) and do NOT require real AWS services. Run these first to verify code correctness.

```
cd /Users/k.nayak.pradeep/Downloads/Kirodemo
pytest -v
```

Expected: All 5 tests pass (test_dynamodb_client.py)

NOTE: If tests fail here, fix code issues before proceeding. No AWS access is needed for this step.

Step 2: Start the Application Server

```
cd /Users/k.nayak.pradeep/Downloads/Kirodemo
uvicorn main:app --reload --port 8000
```

Expected: Console shows "Uvicorn running on http://127.0.0.1:8000"

NOTE: Keep this terminal open. Open a NEW terminal for the following test commands.

Verify the server is running:

```
curl -s http://localhost:8000/docs | head -5
```

Expected: Returns HTML content (FastAPI Swagger docs page)

Step 3: Test - No Contract ID Provided

Simulates a customer who hasn't specified which contract they want.

```
curl -s -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "Hi, I need help"}' | python3 -m json.tool
```

Expected: HTTP 200, status="AUTO", message asks user to provide contract_id, contract_summary=null

AI Banking Assistant - Test Execution Guide

Prerequisites & Step-by-Step Test Instructions

Step 4: Test - Valid Contract ID (Happy Path)

Simulates a customer providing a valid contract ID that exists in DynamoDB.

```
curl -s -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "Show me my contract details", "contract_id": "C123"}' | python3 -m json.tool
```

Expected: HTTP 200, status="AUTO", contract_summary contains AI-generated summary, message contains "C123"

NOTE: This test requires BOTH DynamoDB and Bedrock to be working. If it fails, check AWS credentials and model access.

Step 5: Test - Invalid Contract ID (Human-in-the-Loop Escalation)

Simulates a customer providing a contract ID that does NOT exist. This triggers escalation.

```
curl -s -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "Check my loan", "contract_id": "DOES_NOT_EXIST"}' | python3 -m json.tool
```

Expected: HTTP 200, status="ESCALATE", message="Contract not found, escalating to support", contract_summary=null

Step 6: Test - Missing Required Field (Validation Error)

Simulates a malformed request where the required 'message' field is missing.

```
curl -s -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"contract_id": "C123"}' | python3 -m json.tool
```

Expected: HTTP 422, validation error indicating 'message' field is required

Step 7: Test - Message Exceeds Maximum Length

Simulates a message that exceeds the 1000-character limit.

```
curl -s -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d "{\"message\": \"$(python3 -c "print('A' * 1001)")\"}" | python3 -m json.tool
```

Expected: HTTP 422, validation error indicating string too long (max 1000)

AI Banking Assistant - Test Execution Guide

Prerequisites & Step-by-Step Test Instructions

Step 8: Run Automated Demo Script (Both Use Cases)

Runs both the valid and invalid contract_id scenarios automatically and reports results.

```
cd /Users/k.nayak.pradeep/Downloads/Kirodemo
python demo_use_cases.py
```

Expected: Both use cases pass: Use Case 1 returns AUTO, Use Case 2 returns ESCALATE

Step 9: Verify Structured Logging

Check the terminal where uvicorn is running. You should see JSON-formatted log entries for each request:

```
# Look for entries like:
{"timestamp": "...", "level": "INFO", "message": "Incoming chat request", "contract_id": "C123"}
{"timestamp": "...", "level": "INFO", "message": "Contract retrieved successfully", "contract_id": "C12
{"timestamp": "...", "level": "INFO", "message": "Summary generated successfully"}
{"timestamp": "...", "level": "INFO", "message": "Response sent", "status": "AUTO"}
```

Expected: All log entries are JSON-formatted with appropriate log levels (INFO, WARNING, ERROR)

Troubleshooting Common Issues

Problem: Connection refused on localhost:8000

Solution: Make sure uvicorn is running. Run: uvicorn main:app --reload --port 8000

Problem: HTTP 502 on valid contract_id

Solution: DynamoDB connection failed. Check: aws sts get-caller-identity and verify table exists in us-east-1.

Problem: Summary generation failed (Bedrock error)

Solution: Verify Bedrock model access is enabled. Go to AWS Console > Bedrock > Model access and check Claude 3 Haiku status.

Problem: Import errors when starting server

Solution: Run: pip install -r requirements.txt -r requirements-dev.txt

Problem: Unit tests fail with moto errors

Solution: Ensure moto is installed: pip install moto[dynamodb]. Use moto >= 4.0.